

Technical Support Ticket Management System

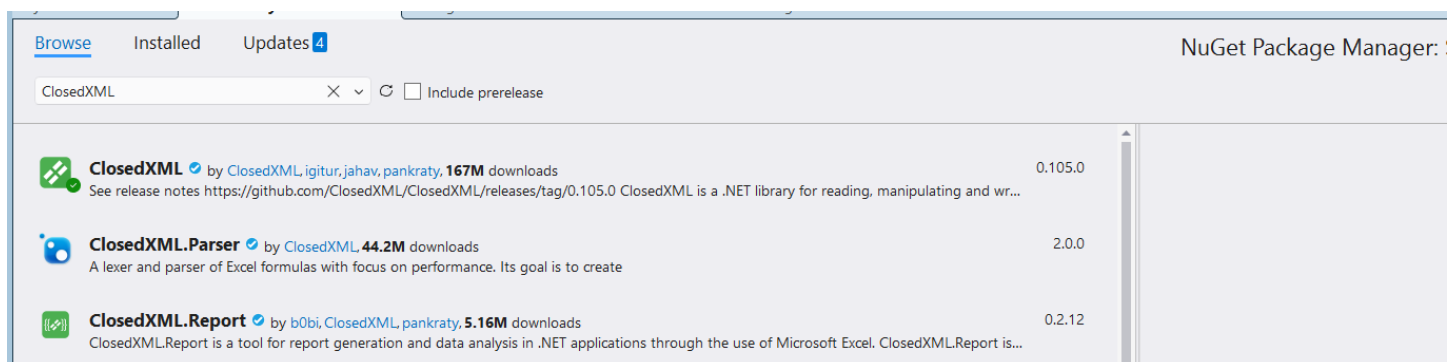
MÓDULO 9 — Generación de Reportes (parte 4, continuación)

Objetivo: Generar dinámicamente un archivo .xlsx con métricas de resolución y permitir su descarga desde el Dashboard del Administrador.

Para lograr exportar datos para análisis externo (SLA, rendimiento de técnicos, picos de demanda), en **.NET 10**, se utiliza la librería **ClosedXML**, que es el estándar de la industria por su facilidad de uso y potencia.

1. Instalación de Dependencias

Primero, se debe instalar el paquete NuGet **ClosedXML** al proyecto desde la terminal o el gestor de paquetes de Visual Studio:



2. Lógica de Cálculo en TicketService.cs

Se implementa un método que procesa los tickets y calcula el **Tiempo de Resolución**. Para este reporte, se considera que un ticket se resuelve en la fecha de su última actualización (UpdatedAt) cuando el estado se actualiza a Resolved.

Archivo: /Services/TicketService.cs

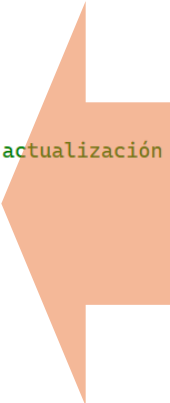
```
1 using ClosedXML.Excel;
2 using Microsoft.EntityFrameworkCore;
3 using SupportTicketsSystem2026.Data;
4 using SupportTicketsSystem2026.Enums;
5 using SupportTicketsSystem2026.Models;
6 using System.Security.Claims;

273 // Método para generar un reporte en Excel con el resumen de tickets, usando ClosedXML
274 1 reference
275 public async Task<byte[]> GenerateExcelReportAsync()
276 {
277     using var context = dbFactory.CreateDbContext();
278     // Traer todos los tickets con sus relaciones para mostrar información completa en el reporte
279     var tickets = await context.Tickets
280         .Include(t => t.Author)
281         .Include(t => t.Technician)
282         .Include(t => t.Category)
283         .ToListAsync();
284     // Crear un nuevo libro de Excel
285     using var workbook = new XLWorkbook();
286     var worksheet = workbook.Worksheets.Add("Resumen de Tickets");
287
288     // Encabezados
289     var headers = new[] { "ID", "Título", "Categoría", "Prioridad", "Estado", "Creado", "Resuelto",
290         "Tiempo (Días)", "Autor", "Técnico" };
291     // Estilo para encabezados
292     for (int i = 0; i < headers.Length; i++)
293     {
294         worksheet.Cell(1, i + 1).Value = headers[i];
295         worksheet.Cell(1, i + 1).Style.Font.Bold = true;
296         worksheet.Cell(1, i + 1).Style.Fill.BackgroundColor = XLColor.LightGray;
297     }
```

```

298 // Llenar datos de los tickets
299 int row = 2;
300 foreach (var t in tickets)
301 {
302     worksheet.Cell(row, 1).Value = t.Id;
303     worksheet.Cell(row, 2).Value = t.Title;
304     worksheet.Cell(row, 3).Value = t.Category?.Name ?? "N/A";
305     worksheet.Cell(row, 4).Value = t.Priority.ToString();
306     worksheet.Cell(row, 5).Value = t.Status.ToString();
307     worksheet.Cell(row, 6).Value = t.CreatedAt;
308     // Solo calcular el tiempo de resolución si el ticket está resuelto y tiene fecha de actualización
309     if (t.Status == TicketStatus.Resolved && t.UpdatedAt.HasValue)
310     {
311         worksheet.Cell(row, 7).Value = t.UpdatedAt.Value;
312         // Cálculo de tiempo de resolución (Diferencia entre fechas)
313         var duration = t.UpdatedAt.Value - t.CreatedAt;
314         worksheet.Cell(row, 8).Value = Math.Round(duration.TotalDays, 2);
315     }
316
317     worksheet.Cell(row, 9).Value = t.Author?.Email;
318     worksheet.Cell(row, 10).Value = t.Technician?.Email ?? "No asignado";
319     row++;
320 }
321 // Ajustar el ancho de las columnas para que se vea bien
322 worksheet.Columns().AdjustToContents();
323 // Guardar el libro en un MemoryStream y retornar el arreglo de bytes
324 using var stream = new MemoryStream();
325 workbook.SaveAs(stream);
326 return stream.ToArray();
327 }
328 }
329 }
330

```



3. Puente de Descarga (JavaScript Interop)

JavaScript Interop (Interoperabilidad de JavaScript) es la capacidad que tienen lenguajes de programación, como C# (en Blazor), para llamar a funciones de JavaScript y viceversa, permitiendo usar librerías JS existentes y APIs del navegador (geolocalización, almacenamiento) directamente en aplicaciones web, aprovechando el ecosistema JS sin reescribir código.

Detalles Clave de JavaScript Interop

- **Bidireccionalidad:** Permite que código no-JS (como C# en Blazor) ejecute funciones JS, y que JS llame a métodos del lado del servidor/cliente.
- **Uso en Blazor:** Se utiliza IJSRuntime para invocar funciones JS desde .NET, comúnmente con métodos como `InvokeAsync<T>`.
- **Casos de uso típicos:** Manipulación del DOM, integración de librerías de gráficos, uso de APIs de navegador (notificaciones, cámara) o librerías JS especializadas.

En nuestro caso, esta tecnología es crucial, Como Blazor Server corre en el servidor, necesitamos un script para que el navegador del cliente reciba el archivo y lo descargue.

Crear el Archivo: [wwwroot/js/downloadHelper.js](#) (Asegurarse de referenciar este script en App.razor).

En el **Explorador de Soluciones**, clic derecho sobre `wwwroot`:

1. **Add -> New Folder ->** Nómbrarlo js.
2. Dentro de js, clic derecho -> **Add -> New Item -> JavaScript File ->** Nómbrarlo `downloadHelper.js`.

```

1 window.downloadFile = (fileName, byteBase64) => {
2     const link = document.createElement('a');
3     link.download = fileName;
4     link.href = "data:application/octet-stream;base64," + byteBase64;
5     document.body.appendChild(link);
6     link.click();
7     document.body.removeChild(link);
8 }

```

4. Interfaz en el Dashboard del Admin

Agregar un botón en el Dashboard para disparar la exportación del archivo.

Editar el Archivo: `/Components/Pages/Admin/Dashboard.razor`

```
1  @page "/admin/dashboard"
2  @using Microsoft.AspNetCore.Authorization
3  @inject TicketService TicketService
4  @inject NavigationManager Navigation
5  @inject IJSRuntime JS
6  @attribute [Authorize(Roles = Roles.Admin)]
7  @rendermode InteractiveServer
8
9  <PageTitle>Admin Dashboard - System Overview</PageTitle>
10
11 <div class="d-flex justify-content-between align-items-center mb-4">
12     <h3><i class="bi bi-speedometer2"></i> Administrative Dashboard</h3>
13     <span class="text-muted">Real-time Support Metrics</span>
14     <button class="btn btn-outline-success" @onclick="ExportToExcel">
15         <i class="bi bi-file-earmark-excel"></i> Exportar Reporte SLA
16     </button>
17 </div>
18
19 <div class="container-fluid mt-4">
```

Agregar el método `ExportToExcel` para exportar el reporte a Excel.

Editar la sección `@code{....}`

```
113 @code {
114     private List<Ticket> allTickets = new();
115     private Dictionary<string, int> stats = new()
116     {
117         ["Total"] = 0,
118         ["Open"] = 0,
119         ["InProgress"] = 0,
120         ["Resolved"] = 0,
121         ["Urgent"] = 0
122     };
123
124     protected override async Task OnInitializedAsync()
125     {
126         // Cargar Lista
127         allTickets = await TicketService.GetAllTicketsAsync();
128
129         // Cargar Estadísticas (Asegúrate de tener este método en TicketService)
130         stats = await TicketService.GetTicketStatsAsync();
131     }
132
133     private string GetStatusBadgeClass(TicketStatus status) => status switch
134     {
135         TicketStatus.Open => "bg-info",
136         TicketStatus.InProgress => "bg-warning text-dark",
137         TicketStatus.Resolved => "bg-success",
138         _ => "bg-secondary"
139     };
140
141     // Método para exportar el reporte SLA a Excel
142     private async Task ExportToExcel()
143     {
144         try
145         {
146             var excelData = await TicketService.GenerateExcelReportAsync();
147             var base64 = Convert.ToBase64String(excelData);
148             var fileName = $"Reporte_Tickets_{DateTime.Now:yyyyMMdd}.xlsx";
149             // Llamar a la función JavaScript para descargar el archivo
150             await JS.InvokeVoidAsync("downloadFile", fileName, base64);
151         }
152         catch (Exception ex)
153         {
154             // Manejar error (notificación visual)
155             Console.WriteLine(ex.Message);
156         }
157     }
158 }
```

5. Referenciar el archivo (Paso Crítico)

Abrir el archivo: Components/App.razor

Añadir la etiqueta <script> justo antes de que cierre el </body>:

```
16
17 <body>
18     <Routes @rendermode="PageRenderMode" />
19     <ReconnectModal />
20     <script src="@Assets["_framework/blazor.web.js"]"></script>
21     <script src="@Assets["Components/Account/Shared/PasskeySubmit.razor.js"]" type="module"></script>
22     @* --- Agregar el Script --- *@
23     <script src="js/downloadHelper.js"></script>
24 </body>
25
```

Actualizar Repositorio (Requerido)

```
Developer PowerShell
+ Developer PowerShell v | | |
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   SupportTicketsSystem2026/Components/App.razor
    modified:   SupportTicketsSystem2026/Components/Pages/Admin/Dashboard.razor
    modified:   SupportTicketsSystem2026/Services/TicketService.cs
    modified:   SupportTicketsSystem2026/SupportTicketsSystem2026.csproj

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    SupportTicketsSystem2026/wwwroot/js/

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git add .
warning: in the working copy of 'SupportTicketsSystem2026/wwwroot/js/downloadHelper.js', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   SupportTicketsSystem2026/Components/App.razor
    modified:   SupportTicketsSystem2026/Components/Pages/Admin/Dashboard.razor
    modified:   SupportTicketsSystem2026/Services/TicketService.cs
    modified:   SupportTicketsSystem2026/SupportTicketsSystem2026.csproj
    new file:   SupportTicketsSystem2026/wwwroot/js/downloadHelper.js

PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git commit -m "Modulo 9(part4): Implementar la logica para la generacion de reportes en Excel"
[main bd72c5d] Modulo 9(part4): Implementar la logica para la generacion de reportes en Excel
 5 files changed, 99 insertions(+), 6 deletions(-)
 create mode 100644 SupportTicketsSystem2026/wwwroot/js/downloadHelper.js
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026> git push origin main
Enumerating objects: 25, done.
Counting objects: 100% (25/25), done.
Delta compression using up to 24 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (14/14), 2.89 KiB | 1.45 MiB/s, done.
Total 14 (delta 8), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (8/8), completed with 8 local objects.
To https://github.com/siciuprp2020/SupportTicketsSystem2026.git
 c40885b..bd72c5d  main -> main
PS C:\Users\luisd\source\repos\SupportTicketsSystem2026>
```